LM Lenguajes de Marcas — Estación 3 de 9 • 1º DAM y DAW

JavaScript y el DOM

Scripts de cliente, fundamentos de JavaScript, el DOM, crear y modificar elementos, eventos y delegación, formularios, fetch y XML; 6 retos autocorregidos (hasta un autocompletado accesible), ejemplo resuelto, 6 ejercicios y test.

120 min lectura · Intermedio · Revisado el 29 de septiembre de 2026

Un proyecto de **Rodrigo Cabello Hernández**, profesor de DAM

Estás aquí

Siguiente estación: **Validación de XML: DTD y XML Schema** Dirección **Ejercicios de Lenguajes de Marcas** →

1. Lenguajes de script de cliente

HTML describe la **estructura** de una página y CSS su **presentación**, pero ninguno de los dos puede reaccionar a lo que hace el usuario. Para eso existen los **lenguajes de script de cliente**: pequeños programas que el navegador descarga junto a la página y **ejecuta en el propio equipo del usuario**. Con ellos se validan formularios antes de enviarlos, se muestran y ocultan elementos, se cargan datos sin recargar la página o se construyen aplicaciones completas como Gmail o Google Docs.

La analogía de la casa

HTML = estructura

Paredes, puertas y ventanas: qué hay y dónde está.

CSS = decoración

Pintura, muebles y colores: cómo se ve.



JavaScript = instalación eléctrica

Interruptores, timbre y domótica: qué pasa cuando actúas.

Cliente frente a servidor

	Script de cliente	**Script de servidor**
Dónde se ejecuta	En el navegador del usuario	En el servidor web, antes de enviar la página
Lenguajes	JavaScript (y lenguajes que se traducen a él: TypeScript), WebAssembly	PHP, Java, Python, Node.js, C#...
Acceso	Al documento (DOM), al navegador y a lo que el usuario permita	A bases de datos, ficheros y servicios del servidor
Seguridad	El usuario puede ver y modificar el código: nunca es la única validación	El código no se ve; es donde se aplican las reglas de verdad
Ejemplos	Menús, pestañas, validación inmediata, gráficos, juegos	Login, pagos, consultas a la base de datos

Lenguajes y tecnologías de cliente

JavaScript: el único lenguaje de script que todos los navegadores ejecutan de forma nativa. Su estándar se llama **ECMAScript** (ES5 en 2009, ES6/ES2015 con let, const, clases y funciones flecha, y una versión nueva cada año).

TypeScript: JavaScript con tipos, creado por Microsoft. El navegador no lo entiende: se **transpila** a JavaScript.

WebAssembly: formato binario al que se compilan C, C++ o Rust para ejecutar código muy rápido en el navegador. Complementa a JavaScript, no lo sustituye.

Históricos: VBScript (solo Internet Explorer), applets de Java, Flash (ActionScript) y Silverlight. Todos han desaparecido por problemas de seguridad y compatibilidad.

Características de JavaScript

Interpretado (en realidad compilado al vuelo por el motor: V8 en Chrome, SpiderMonkey en Firefox, JavaScriptCore en Safari).

Tipado dinámico y débil: una variable puede cambiar de tipo y el lenguaje convierte tipos automáticamente (`"5" * 2` vale 10).

Orientado a eventos: el código reacciona a clics, teclas, cargas de datos...

Multiparadigma: imperativo, orientado a objetos (basado en prototipos, con sintaxis de clases) y funcional.

Se ejecuta en un entorno aislado (sandbox): no puede leer los ficheros del usuario ni acceder a otras pestañas.

Cómo se incluye en una página

index.html

```html
<!-- 1. Fichero externo (lo recomendable): se puede cachear y reutilizar -->
<script src="js/app.js" defer></script>

<!-- 2. Módulo ES: permite import/export y ya se comporta como defer -->
<script type="module" src="js/main.js"></script>

<!-- 3. Script interno (para cosas muy pequeñas) -->
<script>
  console.log("Hola desde la página");
</script>

<!-- 4. En línea en un atributo: desaconsejado, mezcla HTML y lógica -->
<button onclick="alert('Hola')">Saludar</button>
```

Atributo	Descarga	Ejecución	Uso típico
`(ninguno)`	Bloquea el análisis del HTML	Inmediata, en ese punto del documento	Scripts al final del <body>
`defer`	En paralelo	Cuando el HTML está analizado, en orden	La opción por defecto para scripts de la página
`async`	En paralelo	En cuanto llega, sin orden	Scripts independientes (analítica, anuncios)
`type="module"`	En paralelo	Como defer	Código moderno con import/export

¿Por qué defer?

Si un script se ejecuta antes de que exista el HTML que quiere manipular, `document.querySelector` devolverá `null`. Con `defer` (o poniendo el script al final del `<body>`) el documento ya está completo cuando se ejecuta.

▷ Introducción a la programación con JavaScript — midulive — youtube.com/watch?v=Z34BF9PCfYg

2. Fundamentos de JavaScript

Antes de manipular documentos necesitamos la base del lenguaje. Si ya programas en Java verás que la sintaxis es muy parecida (llaves, `if`, `for`, `while`), pero con diferencias importantes en tipos y funciones.

Variables y tipos

variables.js

```js
const PI = 3.1416;        // constante: no se puede reasignar
let edad = 20;            // variable con ámbito de bloque
edad = 21;
// var nombre = "Ana";    // forma antigua: ámbito de función, evítala

// Tipos primitivos
const texto = "Hola";          // string
const numero = 42.5;           // number (enteros y decimales son el mismo tipo)
const activo = true;           // boolean
let sinValor;                  // undefined: declarada pero sin valor
const vacio = null;            // null: "sin valor" asignado a propósito
const grande = 9007199254740993n; // bigint

console.log(typeof texto, typeof numero, typeof activo); // string number boolean

// Plantillas de texto (template literals)
const nombre = "Ana";
console.log(`${nombre} tiene ${edad} años`);
```

> ⚠ **== frente a ===**
>
> `==` convierte los tipos antes de comparar (`"5" == 5` es `true`, `0 == ""` también). Usa siempre `===` y `!==`, que comparan valor **y** tipo.

Control de flujo

control.js

```js
const nota = 7.5;

if (nota >= 9) console.log("Sobresaliente");
else if (nota >= 7) console.log("Notable");
else if (nota >= 5) console.log("Aprobado");
else console.log("Suspenso");

const estado = nota >= 5 ? "aprobado" : "suspenso";   // operador ternario

```

```js
switch (new Date().getDay()) {
  case 0:
  case 6:
    console.log("Fin de semana");
    break;
  default:
    console.log("Día laborable");
}

for (let i = 1; i <= 3; i++) console.log("Vuelta", i);

const modulos = ["Programación", "Bases de Datos", "Lenguajes de Marcas"];
for (const m of modulos) console.log(m);   // for...of recorre valores

let intentos = 0;
while (intentos < 3) intentos++;
```

Funciones

funciones.js

```js
// Declaración: se puede llamar antes de su definición (hoisting)
function sumar(a, b = 0) {           // b tiene valor por defecto
  return a + b;
}

// Expresión de función flecha: más corta, muy usada en eventos y arrays
const cuadrado = (x) => x * x;
const saludar = (nombre) => {
  const mensaje = `Hola, ${nombre}`;
  return mensaje;
};

// Las funciones son valores: se pasan como argumento (callbacks)
function repetir(veces, accion) {
  for (let i = 0; i < veces; i++) accion(i);
}
repetir(3, (i) => console.log("Repetición", i));
```

Arrays y objetos

colecciones.js

```js
const notas = [7, 4.5, 9, 6];
notas.push(8);                              // añadir al final
console.log(notas.length, notas[0]);         // 5 7

const aprobadas = notas.filter((n) => n >= 5);          // [7, 9, 6, 8]
const redondeadas = notas.map((n) => Math.round(n));    // [7, 5, 9, 6, 8]
const media = notas.reduce((suma, n) => suma + n, 0) / notas.length;
const hayDiez = notas.some((n) => n === 10);            // false
```

```
const ordenadas = [...notas].sort((a, b) => a - b);     // copia ordenada

// Objetos: pares clave-valor
const alumno = {
  nombre: "Ana",
  curso: 1,
  modulos: ["LMSGI", "BD"],
  saludar() { return `Soy ${this.nombre}`; },
};
alumno.email = "ana@mail.com";          // añadir propiedad
const { nombre, curso } = alumno;        // desestructuración

// JSON: el formato de intercambio de datos de la web
const json = JSON.stringify(alumno);     // objeto → texto
const copia = JSON.parse(json);          // texto → objeto
```

PRUEBA EL LENGUAJE: LA CONSOLA MUESTRA LOS CONSOLE.LOG

```
const notas = [7, 4.5, 9, 6, 8];
const media = notas.reduce((s, n) => s + n, 0) / notas.length;
console.log("Media:", media.toFixed(2));
console.log("Aprobadas:", notas.filter(n => n >= 5));

const alumno = { nombre: "Ana", curso: 1 };
console.log(`${alumno.nombre} está en ${alumno.curso}º`);
console.log(JSON.stringify(alumno));
```

Consola

```
Media: 6.90
Aprobadas: [7,9,6,8]
Ana está en 1º
{"nombre":"Ana","curso":1}
```

▷ VARIABLES y TIPOS DE DATOS en JAVASCRIPT | ⭐ Curso JAVASCRIPT DESDE CERO #2 — TodoCode — youtube.com/watch?v=kXMpu07zjN4

▷ FUNCIONES en JAVASCRIPT | ⭐ Curso JAVASCRIPT DESDE CERO #23 — TodoCode — youtube.com/watch?v=aYdvt6myUkc

3. El DOM: el documento como árbol de objetos

Cuando el navegador carga una página convierte el HTML en un **árbol de objetos** en memoria: el **DOM** (*Document Object Model*), un estándar del W3C y del WHATWG. Cada etiqueta es un nodo elemento, cada texto un nodo de texto y cada atributo pertenece a su elemento. JavaScript no modifica el fichero HTML: modifica ese árbol, y el navegador vuelve a pintar la página con los cambios.